Forget Gradient Surgery: A Characterization of Projected Gradient Descent for Graph Edge Unlearning

Prabhu Rangaraju[1*]

[1*]Independent Researcher, Illinois, USA. ORCID: 0009-0009-1861-1401.

Corresponding author(s). E-mail(s): prab.rangaraju@gmail.com;

Abstract

Deleting an edge from the adjacency matrix does not erase what the parameters learned from it, and naive gradient ascent on forgotten edges degrades the rest of the graph. We study Forget Gradient Surgery (FGS), an approximate edge-unlearning method: a forget direction (the gradient of the forgotten-edge loss under the post-deletion adjacency) is orthogonally projected against a retain anchor (the link-prediction margin over retained edges), and stopping picks the iterate whose forgotten-edge scores are least distinguishable (Kolmogorov–Smirnov) from never-trained validation edges. The projection primitive is classical; the contribution is the graph-edge formulation, characterized by when it works and when it does not. Against retrain-from-scratch on Cora, CiteSeer, and PubMed ($\boldsymbol{n = 10}$ seeds, two-layer GCN, 1%–20% deletion; link-inference attack AUC for forgetting, link-prediction AUC for utility), the KS-stopped variant runs at a median **0.30–0.48**× retraining time, with no detectable difference from retrain on PubMed (attack gap $\boldsymbol{+0.002}$, $\boldsymbol{p = 0.40}$) and Cora ($\boldsymbol{+0.008}$, $\boldsymbol{p = 0.19}$)—non-significance at this power, not a tested equivalence—but larger, outlier-driven gaps on CiteSeer (median $\boldsymbol{+0.018}$). Against a reimplemented NumPy port of GNNDelete at its untuned setting, FGS forgets closer to retrain on all three datasets; post-hoc tuning of the port on the same seeds lets it match or beat FGS, and a shadow-retrain auditor reopens the CiteSeer closure of attack-calibrated stopping—both are implementation- and auditor-conditional. All evidence is on homophilic citation graphs, one backbone, and a fixed auditor; the characterization does not extend beyond that regime.

Keywords: Graph unlearning, Machine unlearning, Graph neural networks, Projected gradient descent, Edge deletion, Link prediction

1 Introduction

Regulations such as the General Data Protection Regulation (EU 2016/679) grant individuals the right to have their personal data erased on request. In machine learning, this obligation is operationalized as *machine unlearning*: given a model trained on a dataset D and a deletion request for a subset $D_f \subset D$, produce a model that behaves as if D_f had never been part of training. The gold standard is *retraining from scratch* on $D \setminus D_f$; every approximate method is judged by how closely it can reproduce that reference, and at what fraction of the cost [1, 2].

For graph neural networks, the problem is harder than in the i.i.d. setting, and the practical need is pressing. Message passing couples training examples: information from a deleted node or edge propagates into the representations of its neighbors, so "forget" is not a local operation [3, 4]. At the same time, deletion requests arrive continuously; retraining a production GNN on each one is operationally infeasible, and the recent benchmark of Jain et al. [5] finds that most existing graph unlearning techniques are not yet practical on large graphs: certified methods restrict the architecture, shard-based methods assume partitionable training, and several approximate methods lack a calibrated stopping criterion. The gap we address is a *fast, post-hoc, approximate* method for edge deletion with a *data-driven stop*: one that runs in a fraction of retraining time, works on standard nonlinear GNNs, and picks its own stopping point. Among the granularities of deletion (node, edge, feature), we focus on *edge unlearning*, which has immediate practical relevance: social connections, citations, and transactions are edges, and deletion requests frequently concern relationships rather than entities [6].

Why naive approaches fail.

Two generic baselines illustrate the tension. *Fine-tuning* on the retained graph does not genuinely forget: the parameters' residual sensitivity to deleted edges survives additional training on the remaining data. *Gradient ascent* (NegGrad)—maximizing the loss on the deleted set—forgets aggressively but is uncontrolled: because the graph couples examples, ascending on D_f damages predictions on the retained graph as well. Our experiments confirm both pathologies quantitatively (Section 6).

Our contribution.

This paper makes two contributions. First, a method: *Forget Gradient Surgery (FGS)*, a post-hoc, model-agnostic edge-unlearning method for link prediction with three ingredients: (i) a forget direction taken as the gradient of the mean link logit over the deleted edges, evaluated with the adjacency *after* deletion, so the embeddings the gradient flows through no longer route messages along the deleted edges; (ii) a deterministic retain-margin anchor—the gradient of the full link-prediction margin (mean score of all retained edges minus mean score of an equal-sized set of non-edges)—projected out of the forget direction, so retained utility is preserved to first order; and (iii) a data-driven stopping rule that selects the iterate whose forgotten-edge score distribution is least distinguishable, by a two-sample Kolmogorov–Smirnov statistic, from that of never-trained validation edges. Second, a characterization: we map where this construction works, where it breaks, and why—across datasets, deletion fractions,

stopping rules, and a matched no-projection ablation—and we report the failures with the same prominence as the successes.

We are explicit about what is new and what is not. *Orthogonal gradient projection is not new,* and we do not claim it. The projection primitive originates in multi-task learning (PCGrad [7]; GradOPS [8]) and has been applied to unlearning in vision models [9] and language models; most pertinently, Zhou et al. [10] formalize retain-invariance as orthogonality to the retain-gradient subspace and prove the equivalence, which supplies the first-order justification our method relies on. FG-OrIU [11] independently applies gradient orthogonal projection for incremental unlearning of vision models; our construction is the graph-edge counterpart, with the forget gradient defined over link logits and stopping calibrated distributionally. *What is new is the graph-edge formulation:* a forget gradient over link logits under the post-deletion adjacency, a retain anchor defined as a link-prediction margin rather than a classification loss, and stopping calibrated by distributional indistinguishability from never-seen validation edges. To our knowledge, no prior graph-unlearning method uses this construction. Adaptive stopping for unlearning also has precedent outside graphs—MASC [12] introduces an online margin-based stopping rule for LLM unlearning—which we cite to keep our novelty claim properly scoped: FGS's stopping rules are new in mechanism (distributional and attack calibration) and in application (graph edges), not in the bare idea of adapting the stop to the data.

Evaluation criterion.

Following recent benchmark practice [5], we judge unlearning by three desiderata—*forgetting*, *utility*, and *efficiency*—all measured against the retrain-from-scratch reference, and we present the three as a Pareto frontier rather than collapsing them into a single "best" claim. We do *not* require the attack AUC to reach random guessing: absolute link-inference performance stays high even after retraining, because the graph structure itself permits inference of never-trained links [6]. The correct criterion is *indistinguishability from retraining*, which our forgetting metric measures as the gap to retrain's attack AUC).

Results in brief.

On Cora, CiteSeer, and PubMed, FGS with KS stopping stays near retrain on forgetting and utility on PubMed at every tested deletion fraction (5%, 10%, 20%) and on Cora at 5%, while running at 0.30–0.35× retraining time (median); it degrades at 20% deletion on Cora (utility drops of 0.05–0.06; the attack gap there, -0.022, is not distinguishable from zero at $n = 5$) and shows elevated attack gaps on CiteSeer (attack gaps $+0.009$ to $+0.033$ across fractions, $n = 5$ each), where we diagnose a proxy/optimum mismatch between the KS stopping criterion and the attack metric. Run head-to-head on the same protocol and seeds, the GNNDelete deletion-operator baseline forgets less well than FGS on Cora and CiteSeer at its untuned port setting ($\lambda = 1.0$) and over-forgets severely on PubMed; but a λ sensitivity sweep (Section 6.1) shows tuned GNNDelete ($\lambda = 0.5$, a post-hoc selection over five λ values on the same ten seeds) matches or exceeds FGS on forgetting on Cora and CiteSeer at roughly half the cost—so the frontier FGS traces is partly covered by a tuned existing baseline, and

FGS's distinctive contribution is the KS-stopped operating point (untuned, 0.30–0.35× cost) plus the honest map of where each method breaks. An unlink-only baseline falls short of retrain indistinguishability on Cora and CiteSeer (but not PubMed, where structural deletion does most of the work). An attack-calibrated stopping variant closes the CiteSeer gap under the evaluation auditor (a shadow-retrain auditor does not reproduce the closure; see Limitations) but costs 0.49–0.74× retraining time (median 0.63×); a lower-cost hybrid variant recovers a significant share of the gain at near-KS cost (paired improvement over KS +0.010, $p < 0.001$). A matched no-projection ablation (same attack-calibrated stopping, projection removed) isolates the projection's causal contribution: without it, utility sits 4–5 percentage points below retrain on Cora and CiteSeer. We also report the negatives: a margin-guarded stopping variant that fails to improve on plain KS, and a single-seed GraphSAGE transfer that is suggestive but not generalization evidence.

2 Related Work

Graph unlearning families.

Recent surveys and benchmarks organize graph unlearning into a handful of families [4, 5, 13]. *Shard-based (SISA-style)* methods such as GraphEraser [3] and GUIDE [14] partition the graph and retrain only affected shards; they are exact only under a per-shard model and suffer partition overhead on non-shardable graphs. *Influence-function* methods such as GIF [15] approximate the parameter change of deletion via the Hessian, which is expensive and rests on convexity or convergence assumptions. *Learning-based* methods include GNNDelete [16], which inserts trainable deletion operators and evaluates node, edge, and feature deletion, and MEGU [17], which runs a mutual-evolution loop between prediction and unlearning modules, and SURGE [18], which guides residual distillation with a structural perturbation response field. *Projection/closed-form* approaches include PROJECTOR [19], which projects linear-GNN weights into a subspace orthogonal to the deleted data (with exactness guarantees for its custom linear architecture), and ETR [20], which edits parameters by masking those critical to the unlearned samples and then rectifies utility with a subgraph gradient approximation. *Corrective unlearning*, e.g. COGNAC [21], addresses manipulated rather than deleted data. FGS differs from all of these: it is post-hoc on a single model (no sharding), Hessian-free (no convexity assumptions), requiring no new trainable modules, and applicable to standard nonlinear GNNs (unlike PROJECTOR's linear setting).

Edge unlearning specifically.

GNNDelete frames edge deletion through its deletion-operator layers and reports Deleted Edge Consistency gains on edge-deletion tasks, and recent work shows that deleting the forget edges from the adjacency alone (*Unlink to Unlearn*, UtU [6]) can already remove much of their influence—because GNN parameters concentrate in the feature-update step, edges act primarily as message-passing conduits. FGS builds on this insight but goes further: it adjusts the parameters so that the model's *scores* on

the forgotten edges also align with those of a retrained model, which mere unlinking does not address.

Gradient projection and unlearning.

Orthogonalizing conflicting gradients is a multi-task-learning primitive: PCGrad [7] projects each task's gradient onto the normal plane of conflicting ones, and GradOPS [8] projects onto subspaces spanned by all tasks. For unlearning, Hoang et al. [9] proposed projected-gradient unlearning for vision DNNs (projecting the forget gradient against retain gradients), and OrthoGrad [22] uses per-sample gradient orthogonalization via QR decomposition. The closest precedent for our justification is Zhou et al. [10], who formalize "no side effects" as local retain invariance under small parameter updates and *prove* it holds if and only if the update direction is orthogonal to the subspace spanned by retain gradients—the same first-order argument our Proposition 1 makes for the special case of a scalar retain margin. Their method targets LLM unlearning (TOFU/MUSE/WMDP); FGS is the graph-edge instantiation, with a link-prediction-margin anchor and calibrated stopping that GU lacks. We credit the primitive and claim only the formulation.

Adaptive stopping for unlearning.

Stopping rules are the least standardized part of approximate unlearning: most methods run a fixed budget. The clearest precedent for adapting the stop to the data is MASC [12], which returns the first checkpoint at which a monitored margin-violation rate falls below tolerance, explicitly contrasting this with fixed-compute-budget methods. MASC operates on LLM forget sequences with a margin statistic; FGS's rules are distributional (KS against never-seen validation edges) and attack-calibrated (ranking alignment), and they operate on graph edges. The idea of adaptive stopping is therefore precedented; our stopping mechanisms and their graph application are new. The positioning table (Table 1) marks which *graph* methods decide their own stopping point.

Evaluating edge unlearning.

Attack-based evaluation treats unlearning as a privacy game: can an adversary distinguish forgotten items from never-seen ones? Link-inference ("link stealing") attacks on GNNs are the natural auditor for edge unlearning [6, 23]. Certified unlearning frameworks with theoretical guarantees have been developed for the graph setting, e.g., IDEA [24], which covers node, edge, and attribute requests; FGS offers no such certificate, and we do not claim one. The ICLR-2026 benchmark of Jain et al. [5] structures evaluation around efficiency, utility, and forgetting, and finds most existing techniques not yet practical on large graphs—the standard FGS is held to.

A dedicated benchmark for edge unlearning, Lethe, was accepted to the NeurIPS 2026 Experiments & Benchmarks track [25]; its technical manuscript was not publicly available at the time of writing, so no comparison with it was possible; we note it as contemporaneous work rather than citing results from it.

Table 1 Positioning of FGS among graph unlearning methods. "Adaptive stop" indicates whether the method itself decides how much to forget; "nonlinear" indicates applicability to standard nonlinear GNNs. Adaptive stopping for unlearning is precedented outside graphs (MASC [12]); the column is scoped to graph methods. "Tested" marks the baselines we run head-to-head in Section 6.

Method	Mechanism	Guarantee	Nonlinear / granularity	Adaptive stop	Tested
GraphEraser [3]	shard partition, retrain affected	exact (per shard)	yes / node, edge	no	no
GNNDelete [16]	trainable deletion operators	approximate	yes / node, edge, feature	no	yes
GIF [15]	influence-function Newton update	approximate	yes / node, edge	no	no
MEGU [17]	mutual-evolution unlearning	approximate	yes / node, edge	partial	no
PROJECTOR [19]	closed-form projection	exact	no (linear GNNs) / node	n/a	no
UtU [6]	unlink (structural only)	approximate	yes / edge	n/a	yes
ETR [20]	training-free parameter edit	approximate	yes / node, edge	no	no
FGS (this work)	projected gradient + calibrated stopping	approximate	yes / edge	yes	—

Exact vs. approximate, certified vs. empirical.

A recurring axis in the literature is whether an unlearning method offers *exactness* (the unlearned model equals retraining in distribution) or *certification* (a provable bound on residual information). Shard-based methods are exact only under a per-shard model, which real-world graphs rarely admit; certified approaches achieve their guarantees by restricting to custom linear GNNs where Hessians stay tractable, and, as the recent benchmark observes, offer limited practical value on the nonlinear architectures used in practice [5]. FGS sits firmly on the approximate, empirical side: it provides no certificate, and its evidence is indistinguishability under a fixed auditor. We view this as the honest position for a post-hoc method on standard nonlinear GNNs, and we scope our claims accordingly.

Gradient-based unlearning beyond graphs.

The forget/retain tension FGS navigates appears across unlearning literature: gradient ascent on the forget set (NegGrad) is the oldest baseline and is known to damage retained performance without additional control; subsequent work adds retain-set regularization, KL constraints, or projection. LUR [26] tackles the same forget/retain gradient conflict directly, via an implicit regularization that steers optimization toward a conflict-free direction rather than an explicit projection. FGS's specific control—orthogonal projection against a deterministic full-set margin gradient—is chosen for the link-prediction setting, where the natural retain quantity is a score margin rather than a classification loss.

3 Preliminaries

Notation.

Let $G = (V, E, X)$ be an undirected graph with $|V| = n$ nodes, edge set E, and node features $X \in \mathbb{R}^{n \times d}$. Let $A \in \{0,1\}^{n \times n}$ be the adjacency matrix and $\tilde{A}$ its symmetrically normalized form with self-loops. A two-layer GCN [27] encoder produces embeddings $Z = f_\theta(X, \tilde{A}) \in \mathbb{R}^{n \times h}$; the link logit for a pair (u, v) is the dot product $s_{uv}(\theta) = z_u^\top z_v$. Let E_{tr} be the training edges, $E_f \subset E_{\text{tr}}$ the deleted (forget) set, $E_r = E_{\text{tr}} \setminus E_f$ the retained training edges, and A_r the adjacency after deleting E_f. Let N_r be a set of non-edges with $|N_r| = |E_r|$.

Problem.

Given parameters $\theta^\star$ trained on (X, A) with edges E_{tr}, and a deletion request E_f, produce θ^u such that a link-inference adversary cannot distinguish E_f from never-trained edges any better than it can against θ^{re}, the parameters obtained by retraining from scratch on (X, A_r) with edges E_r, while preserving link-prediction utility on held-out edges and costing less than retraining.

Threat model / auditor.

We use a white-box, non-adaptive auditor: it scores candidate pairs with the model's own link logits $s_{uv}(\theta)$, and we report the AUC of separating E_f from hard non-edges matched on label-pair structure. *Lower attack AUC means better forgetting;* the reference is the retrained model's attack AUC, not 0.5. The auditor is fixed before unlearning and does not adapt (e.g., no shadow models); see Section 5.

Attack-negative sampling.

"Hard" means each non-edge is matched on the sorted label pair of its forget edge: for a forget edge between a class-a and class-b node, the negative is drawn uniformly from non-edges whose endpoints have labels $\{a, b\}$. Random non-edges are a weak test—most are cross-community pairs the model scores near zero regardless of training, so the attack AUC saturates near 1.0 for every method and reveals nothing. Label-pair matching forces the negatives to be distributionally close to the forget edges, so the AUC measures whether the model specifically retains the deleted edges' signal rather than generic graph structure. The stopping rule uses a separately sampled hard-negative set, disjoint from the evaluation negatives (Section 4.4).

3.1 Why edge deletion is nonlocal: the blast radius

In an L-layer message-passing GNN, the embedding z_v depends on the L-hop neighborhood of v. Deleting a single edge (u, v) therefore changes the *computation graph* of every node within L hops of u or v—the deletion's *blast radius* (the set of node representations whose receptive field includes the deleted edge). On Cora with $L = 2$, we computed the 2-hop blast radius over 500 sampled training edges: median 57 nodes (mean 95, maximum 542 of 2,708 nodes), so a single deletion touches the receptive fields of dozens to hundreds of nodes depending on local density. Two consequences follow. First, the trained parameters $\theta^\star$ were optimized while messages flowed along E_f; they encode co-occurrence patterns involving the deleted edges that persist after the adjacency is corrected. We hypothesize that unlinking alone is therefore insufficient for parametric indistinguishability from retraining. We test this directly with an unlink-only baseline (UtU): the base model, trained *with* the forget edges, evaluated on the post-deletion adjacency A_r with no parameter change. Unlinking alone leaves attack gaps of $+0.058$ on Cora and $+0.105$ on CiteSeer (both $p < 0.001$, $d_z = 2.72/1.94$), far from retrain indistinguishability, confirming that the parameters' residual memory of the deleted edges survives structural deletion and that the surgery's parametric adjustment is doing real work beyond the adjacency fix. On PubMed, the unlink-only gap is small ($+0.009 \pm 0.004$)—there the graph structure itself carries little forget-edge signal

beyond retraining—so the headroom FGS adds is correspondingly smaller. Second, any gradient step computed on E_f—including naive gradient ascent—backpropagates through the same blast radius, perturbing predictions on retained edges whose receptive fields overlap it. The surgery in Section 4 is designed for exactly this situation: it permits movement along the forget direction only in the subspace orthogonal to the retain-margin gradient.

Table 2 Notation.

Symbol	Meaning
$G = (V, E, X)$, A, A_r	graph, adjacency, post-deletion adjacency
E_{tr}, E_f, E_r	train / forget / retained edges
N_r, E_v	retained non-edges; validation edges (never trained on)
f_θ, z_v, s_{uv}	GCN encoder, embedding, link logit
g_f, g_r, g	forget / retain-margin / projected gradient
$M(\theta)$, $F(\theta)$	retain margin; mean forget-edge logit
θ^{re}, θ^u	retrained / unlearned parameters
Δ_{atk}	attack-AUC gap to retrain (forgetting metric)

Remark 1 (Why absolute attack AUC stays high) Even the retrained model scores ≈ 0.92 attack AUC on PubMed: edges absent from training are still inferable from graph structure (triangles, homophily). Demanding attack AUC ≈ 0.5 would demand the model unlearn the graph itself. The meaningful target is the retrain gap, $\Delta_{\text{atk}} = \text{AUC}_{\text{atk}}(\theta^u) - \text{AUC}_{\text{atk}}(\theta^{\text{re}})$.

4 Forget Gradient Surgery

4.1 The two gradients

Forget gradient.

The direction that *raises* forgotten-edge scores is

$$g_f \;=\; \nabla_\theta \underbrace{\frac{1}{|E_f|} \sum_{(u,v)\in E_f} s_{uv}(\theta)}_{\text{mean forget-edge logit}}, \tag{1}$$

computed with embeddings from the *post-deletion* adjacency A_r. Because A_r no longer routes messages along E_f, g_f captures the residual parametric memory of the deleted

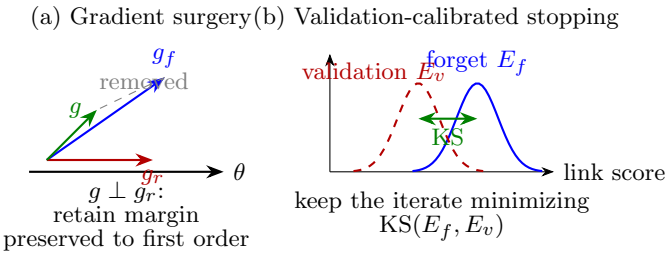


Fig. 1 (a) The forget gradient g_f is projected onto the hyperplane orthogonal to the retain-margin gradient g_r; the update g lowers forget-edge scores without harming the margin to first order. (b) Stopping selects the iterate whose forget-edge score distribution is least distinguishable (two-sample KS) from never-trained validation edges.

edges rather than their structural role. In plain terms: g_f points *uphill* on the forgotten edges' predicted likelihood, so we step *downhill* along it—each update nudges the model to lower those edges' scores.

Retain-margin anchor.

The direction that *raises* the retained link-prediction margin is

$$g_r \;=\; \nabla_\theta \left[\frac{1}{|E_r|} \sum_{(u,v)\in E_r} s_{uv}(\theta) \;-\; \frac{1}{|N_r|} \sum_{(u,v)\in N_r} s_{uv}(\theta) \right], \tag{2}$$

again under A_r. Unlike a classification loss on retained nodes, this margin is the exact quantity link-prediction utility depends on; unlike a minibatch estimate, it uses the *complete* retained edge set with an equal-sized non-edge set, making it a deterministic anchor with no subsampling noise.

4.2 Surgery and normalized descent

The surgically corrected update removes from the forget direction any component that would harm the retain margin:

$$g \;=\; g_f \;-\; \frac{g_f^\top g_r}{\|g_r\|^2}\, g_r, \qquad \theta \;\leftarrow\; \theta \;-\; \eta\, \frac{g}{\|g\|}, \tag{3}$$

with step size $\eta = 0.05$ in our experiments. By construction $g^\top g_r = 0$: to first order, each step leaves the retain margin unchanged while decreasing the mean forget-edge logit. This is precisely the failure mode of naive gradient ascent (NegGrad), which descends along g_f unprojected and therefore erodes E_r utility whenever g_f and g_r conflict—the typical case on real graphs.

4.3 First-order analysis

Let $M(\theta) = \frac{1}{|E_r|}\sum_{E_r} s_{uv} - \frac{1}{|N_r|}\sum_{N_r} s_{uv}$ be the retain margin, so that $g_r = \nabla M(\theta)$, and let $F(\theta) = \frac{1}{|E_f|}\sum_{E_f} s_{uv}$ be the mean forget-edge logit, so that $g_f = \nabla F(\theta)$.

Proposition 1 *For the update (3) with $\eta > 0$ small, and assuming $g \neq 0$:*

(i) the retain margin is stationary to first order, $M(\theta - \eta g/\|g\|) = M(\theta) + o(\eta)$;
(ii) the mean forget-edge logit strictly decreases to first order, $F(\theta - \eta g/\|g\|) = F(\theta) - \eta\,\|g\| + o(\eta)$;
(iii) $g_f^\top g = \|g\|^2 \geq 0$, with equality only if g_f is parallel (or anti-parallel) to g_r.

Proof (i) By Taylor expansion, $M(\theta - \eta g/\|g\|) = M(\theta) - \eta\,\nabla M(\theta)^\top g/\|g\| + o(\eta)$, and $\nabla M(\theta)^\top g = g_r^\top g = 0$ by construction of the projection. (ii) is the same expansion applied to F. (iii) follows from $g_f^\top g = g_f^\top g_f - (g_f^\top g_r)^2/\|g_r\|^2 = \|g_f\|^2 - \|P_{g_r} g_f\|^2 = \|g_f - P_{g_r} g_f\|^2 = \|g\|^2$, where P_{g_r} is the orthogonal projector onto g_r. □

The proposition is worth reading for what it does and does not promise. Part (i) is the precise sense in which the surgery protects utility: no first-order harm to the retain margin, unlike NegGrad. The argument is a special case of the general retain-invariance theorem of Zhou et al. [10], who prove that a small update leaves the retain loss invariant if and only if it is orthogonal to the subspace spanned by retain gradients; we credit that result and claim only its application to the link-prediction margin. Part (iii) shows the forget direction is never reversed by the projection ($g_f^\top g \geq 0$), so forgetting proceeds monotonically to first order—except in the degenerate case where g_f is parallel (or anti-parallel) to g_r, i.e., forgetting the deleted edges and preserving the margin are the *same* direction, in which case $g = 0$ and the normalized step is undefined (in practice we skip the update). The analysis is first-order and local: over many steps the anchor g_r itself moves, which is why the empirical trajectory—not the single-step calculation—is what Section 6 evaluates. This is a local calculation, not an unlearning guarantee: it says nothing about how far the trajectory can travel before the anchor moves, which is exactly what breaks at 20% deletion on Cora (Section 6.2).

4.4 Calibrated stopping rules

A fixed iteration budget is a per-dataset hyperparameter in disguise. FGS instead runs a projected trajectory and keeps one iterate selected by a *data-driven* rule computed on validation edges, which were never trained on and are absent from A_r—the same informational position forgotten edges occupy under retraining. Validation edges are a proxy, not a perfect stand-in: the base model was trained on the same graph, so validation-edge endpoints carry structural signal a retrained model never sees (cf. Remark 1). The KS rule therefore targets distributional similarity under this proxy; Section 6.4 shows where the proxy diverges from the attack optimum. We use only validation data; attack negatives and test edges are never touched during unlearning: E_v is disjoint from the test split and from the evaluation hard-negative set. We test five variants, summarized in Table 3:

Table 3 The five stopping variants at a glance. "Cost" is the stopping overhead beyond the shared projected trajectory.

Variant	Selection rule	Cost
`fgs` (KS)	iterate minimizing KS(forget scores, validation scores)	40-step trajectory
`fgs_margin` (KS+margin)	KS minimum among iterates with mean(forget) $\leq$ mean(validation)	40-step trajectory
`fgs_attack` (attack-calibrated)	iterate minimizing |AUC(forget vs stop-neg) − AUC(validation vs stop-neg)|	80-step trajectory + scoring
`fgs_hybrid` (hybrid)	attack-calibrated key on the five lowest-KS iterates	40-step trajectory + 5 scorings
`fgs_noproj` (no-proj.)	attack-calibrated stopping with the projection removed ($g = g_f$)	80-step trajectory + scoring

KS rule (`fgs`).

Run up to $T_{\max} = 40$ projected steps; keep the iterate minimizing the two-sample Kolmogorov–Smirnov statistic [28] between the forgotten-edge scores and the validation-edge scores. The KS minimum is a heuristic proxy for "looks retrained": if the forgotten-edge scores were drawn from the same distribution as the validation-edge scores, a membership-style adversary could not distinguish them. We use the KS statistic rather than a mean gap because indistinguishability is a distributional property: means can match while variances differ, leaving the adversary signal. Matching the retrain gap under this fixed auditor is an empirical indistinguishability claim, not a privacy guarantee: a stronger or adaptive auditor could still tell the unlearned model apart from retraining.

Margin-guarded KS (`fgs_margin`).

A tried fix for early KS alignment: restrict the KS minimization to iterates satisfying $\text{mean}_{E_f} s \leq \text{mean}_{E_v} s$, falling back to the maximum-forgetting iterate if none qualify. This is a non-dataset-specific rule: no threshold is tuned, only the inequality direction is fixed. It fails—the guard essentially never binds, and the variant is statistically identical to plain KS (Section 6). We keep it as a negative result.

Attack-calibrated rule (`fgs_attack`).

Run up to 80 projected steps; keep the iterate minimizing |AUC(forget vs held-out stop negatives) − AUC(validation vs held-out stop negatives)|, where the stop negatives are freshly sampled hard negatives disjoint from the evaluation attack negatives. This directly calibrates the forget-vs-nonedge *ranking*—what the membership-inference-style attack measures [29]—to the unseen-edge ranking. It explicitly targets attack-style ranking and should be read as such: a validation-calibrated rule, not a test-set-tuned one,

Algorithm 1: Forget Gradient Surgery (FGS)

Input: trained θ, post-deletion adjacency A_r, forget edges E_f, retained edges E_r, non-edges N_r, validation edges E_v, budget $T_{\max}$, step size η
Output: unlearned parameters $\theta_{k^\star}$

$\theta_0 \leftarrow \theta$
$d_0 \leftarrow \mathrm{KS}\big(\{s_{uv}(\theta_0)\}_{E_f}, \{s_{uv}(\theta_0)\}_{E_v}\big)$; $\quad k^\star \leftarrow 0$; $\quad d^\star \leftarrow d_0$
for $t = 1$ **to** $T_{\max}$ **do**
  compute g_f by (1) and g_r by (2) under A_r
  $g \leftarrow g_f - (g_f^\top g_r / \|g_r\|^2)\, g_r$
  $\theta_t \leftarrow \theta_{t-1} - \eta\, g/\|g\|$
  $d_t \leftarrow \mathrm{KS}\big(\{s_{uv}(\theta_t)\}_{E_f}, \{s_{uv}(\theta_t)\}_{E_v}\big)$
  if $d_t < d^\star$ **then** $d^\star \leftarrow d_t$; $\; k^\star \leftarrow t$
end for
return $\theta_{k^\star}$

Fig. 2 Forget Gradient Surgery with KS stopping. The iterate-0 evaluation (d_0, $k^\star \leftarrow 0$) is essential, not incidental: the implementation scores the post-deletion model before any gradient step, and the KS minimum can occur there (e.g., PubMed seed 7), in which case FGS returns the model unchanged.

and not a purely unsupervised stopping rule. It removes the speed advantage (all 80 candidates are evaluated) but substantially closes the CiteSeer attack gap (Section 6).

Hybrid rule (`fgs_hybrid`).

A lower-cost attack-calibrated variant: run the 40-step KS trajectory, keep the five iterates with the lowest KS values, and apply the attack-calibrated key to those five only. This preserves the ranking calibration where the KS proxy is weakest while avoiding the 80-candidate evaluation cost of `fgs_attack`.

4.5 Complexity

Each step performs one forward pass and two backward passes (one for g_f, one for g_r) over $|E_f| + |E_r| + |N_r|$ scored pairs; with $|E_f| \ll |E_r|$ this is $\mathcal{O}(T_{\max} \cdot (|E_r| + |N_r|))$ score evaluations and no Hessian, no sharding, no added parameters, and no retraining. We report wall-clock cost in Section 6.

5 Experimental Setup

Datasets and task.

We use Cora, CiteSeer, and PubMed [30] with the standard link-prediction split: edges are divided 70/15/15 into train / validation / test; the train split is further divided into a forget set E_f (a random fraction p of train edges) and a retained set E_r. Validation edges E_v are never trained on and never part of A_r. Backbone: a two-layer GCN (32-dimensional hidden layer, ReLU) trained with Adam (10^{-2}, 150 epochs, full batch) on link prediction with dot-product logits. All base models use identical initializations and data splits for every method.

Methods.

Retrain (gold standard), *Fine-tune* (extra training on (X, A_r)), *NegGrad* (gradient ascent on E_f), *GNNDelete*—a faithful NumPy port of the deletion-operator method of Cheng et al. [16] to edge unlearning (trainable per-layer linear operators on the forget-edge endpoints, trained for 40 epochs with a deleted-edge-consistency loss toward the non-edge class plus a neighborhood-influence loss matching the base model's scores, with the neighborhood-influence weight fixed at $\lambda = 1.0$; a λ sensitivity sweep is reported in Section 6.1 and the port is not validated against published GNNDelete edge-deletion numbers; documented in the reproduction package)—and *UtU*, the unlink-only baseline: the base model (trained *with* the forget edges) evaluated on the post-deletion adjacency A_r, with no parameter change. Then FGS with the stopping variants of Section 4.4: `fgs` (KS), `fgs_margin` (margin-guarded KS), `fgs_attack` (attack-calibrated), `fgs_hybrid` (hybrid, CiteSeer only), and a no-projection ablation `fgs_noproj` that runs the identical code path with the projection removed ($g = g_f$) and the same attack-calibrated stopping as `fgs_attack`—so the `fgs_attack` vs. `fgs_noproj` comparison, differing only in the projection, is the causal test of the surgery.

Metrics.

Forgetting: attack AUC (link-inference separating E_f from hard non-edges), reported as the gap to retrain, $\Delta_{\text{atk}} = \text{AUC}_{\text{atk}}(\theta^u) - \text{AUC}_{\text{atk}}(\theta^{\text{re}})$; zero means indistinguishable from retraining. The sign is read in words: a positive attack gap means the unlearned model is *more* vulnerable than retrain (under-forgetting); a negative gap means it is *less* vulnerable (over-forgetting). *Utility:* test-set link AUC, reported as the signed gap to retrain, $\Delta_{\text{util}} = \text{AUC}_{\text{util}}(\theta^u) - \text{AUC}_{\text{util}}(\theta^{\text{re}})$ (retrain test AUC is 0.774/0.794/0.939 on Cora/CiteSeer/PubMed, so the gap is the meaningful quantity). *Efficiency:* wall-clock time relative to retraining on identical hardware (2 CPUs, no GPU).

Protocol.

Main sweep: 5% forget fraction on three datasets × ten seeds $(42, 7, 123, 2024, 31337, 999, 2025, 5555, 7777, 31338)$. Fraction sensitivity: 1%, 10%, 20% on five seeds $(42, 123, 2024, 2025, 999)$—a fixed subset of the ten main seeds, held constant across all fractions. Randomness: seed controls initialization, the forget-set draw, and the retain-non-edge draw N_r. Retrain uses the same seed. Statistics: paired (within-seed) comparisons with paired two-sided t-tests; d_z is Cohen's paired effect size. Retrospectively, with $n = 10$ paired seeds, the design detects Cohen's paired $d_z \approx 1.0$ at 80% power ($\alpha = 0.05$, two-sided); the decisive headline effects exceed this bar, while the marginal untuned FGS-vs-GNNDelete gaps ($d_z \approx -0.75$) fall below it and are reported as directional only. All reported means are ± one standard deviation across seeds. For the $n = 5$ fraction-sensitivity comparisons, where skew is a concern, we also report Wilcoxon signed-rank p-values. We do not apply a multiple-comparison correction; the decisive claims (attack-gap closure, utility preservation, ablation utility loss) are all $p < 0.001$, far below any conventional corrected threshold, and we flag the $n = 5$ exploratory comparisons as such rather than leaning on uncorrected p-values. Throughout, we treat any p between 0.01 and 0.05 as exploratory evidence only; no narrative claim rests on one.

Threats to validity and scope.

(i) The auditor is fixed and non-adaptive; stronger adaptive attackers (e.g., shadow models) could find residual signal we cannot see. One such auditor (a per-seed shadow retrain) is tested in Section 8. We make no certified-unlearning claim. (ii) We scope to edge deletion on homophilic citation graphs with a GCN; node and feature deletion are out of scope. (iii) Absolute attack AUC stays high even after retraining (Remark 1); our metric is the retrain gap, which is the correct target but means "forgotten" does not mean "unpredictable." (iv) FGS touches only the parameters, not the adjacency: structural deletion (Unlink) is a separate, complementary step. (v) One GraphSAGE transfer configuration is reported; it is a smoke test, not generalization evidence.

6 Results

6.1 Main comparison: 5% deletion, ten seeds

At 5% deletion, FGS with KS stopping (`fgs`) is the closest to retraining among the approximate methods on all three datasets. Table 4 reports attack and utility gaps to retrain with paired statistics ($n = 10$ seeds). The 95% CIs (paired) for the KS-stopped FGS attack gaps are $[-0.005, +0.020]$ on Cora, $[-0.008, +0.081]$ on CiteSeer, and $[-0.003, +0.006]$ on PubMed. We read these against a ± 0.02 AUC equivalence margin: 0.02 is about 2% of the retrain attack-AUC scale (≈ 0.9) and sits well below the gaps that mark genuine failures in this study (fine-tuning leaves $+0.09$–$+0.12$; NegGrad costs -0.06–-0.09 utility), so containment under it is a demanding bar. Against this margin, only PubMed's interval is fully contained, Cora's straddles the margin's upper edge, and CiteSeer's lies outside it on the high side. The margin itself is author-chosen, so the Cora containment reading is margin-sensitive rather than absolute. We perform no equivalence test (e.g., TOST), and non-significance at $n = 10$ must not be read as equivalence: the design detects only large paired effects ($d_z \approx 1.0$ at 80% power; Section 5). On CiteSeer, the FGS attack gap is not significant at $n = 10$ (paired vs. retrain $p = 0.09$, $d_z = 0.59$), and the mean is driven substantially by a single outlier seed ($+0.181$; dropping it moves the mean to $+0.021$; see Section 7 for the leave-one-out robustness analysis).

Three patterns held up across the sweep:

1. Fine-tuning does not forget; ascent destroys utility.

Fine-tuning on the retained graph leaves large attack gaps: $+0.090$ on Cora and $+0.116$ on CiteSeer (both $p < 0.001$, $d_z = 3.69/2.17$), confirming that residual parametric memory of the deleted edges survives additional training. NegGrad reduces the attack gap but at a steep utility cost: -0.085 on Cora, -0.079 on CiteSeer, -0.062 on PubMed (all $p < 0.001$, $|d_z| > 2.4$), consistent with the blast-radius analysis—unprojected ascent on E_f backpropagates through the receptive fields of retained edges. The GNNDelete deletion-operator baseline (our NumPy port, not validated against published GNNDelete numbers—see Section 5) sits between the two: on Cora its attack gap is $+0.022$ ($p = 0.011$, $d_z = 1.01$) and on CiteSeer $+0.052$ ($p = 0.015$, $d_z = 0.95$)—materially better than fine-tuning but roughly 2–3$\times$ the FGS gap—while on PubMed it

Table 4 5% deletion, ten seeds per dataset: attack gap Δ_{atk} and utility gap Δ_{util} to retrain (mean $\pm$ SD; 0 = identical to retrain), with paired two-sided t-test p-values vs. retrain. On each dataset, seeds 7, 42, 123 come from an earlier run and the other seven from a later run under the same protocol. Exchangeability is established by re-running the three earlier seeds per dataset under the frozen pipeline: every method's attack/util reproduces within 3×10^{-3} (floating-point noise), confirming identical code paths, splits, and hyperparameters. As a supplementary sanity check, a batch indicator (earlier vs. later seeds) on the paired gaps is null for every method and dataset (Welch $p > 0.16$ throughout; CiteSeer `fgs` +0.037 vs. +0.037). All comparisons are within-seed paired, so the pooled $n = 10$ stands. Sign guide: $\Delta_{\text{atk}} > 0$ means the unlearned model is *more* vulnerable than retrain (under-forgetting, worse); $\Delta_{\text{atk}} < 0$ means *less* vulnerable (over-forgetting); $\Delta_{\text{util}} < 0$ is a utility loss (worse). The `fgs_hybrid` row reports the CiteSeer attack gap only (mean +0.027 over the same ten seeds; per-seed values in Table 6); the hybrid was evaluated on CiteSeer alone.

	Cora		CiteSeer		PubMed	
Method	Δ_{atk}	Δ_{util}	Δ_{atk}	Δ_{util}	Δ_{atk}	Δ_{util}
Fine-tune	$+0.090 \pm 0.024$	-0.013 ± 0.023	$+0.116 \pm 0.053$	-0.001 ± 0.026	$+0.003 \pm 0.003$	-0.003 ± 0.002
NegGrad	$+0.012 \pm 0.054$	-0.085 ± 0.025	$+0.005 \pm 0.058$	-0.079 ± 0.032	-0.068 ± 0.014	-0.062 ± 0.017
GNNDelete	$+0.022 \pm 0.022$	-0.017 ± 0.022	$+0.052 \pm 0.055$	-0.004 ± 0.021	-0.085 ± 0.006	-0.007 ± 0.002
UtU (unlink only)	$+0.058 \pm 0.021$	-0.015 ± 0.023	$+0.105 \pm 0.054$	-0.003 ± 0.022	$+0.009 \pm 0.004$	$+0.004 \pm 0.002$
`fgs` (KS)	$+0.008 \pm 0.017$	-0.020 ± 0.023	$+0.037 \pm 0.062$	-0.011 ± 0.023	$+0.002 \pm 0.006$	-0.000 ± 0.004
`fgs_margin` (KS+margin)	$+0.008 \pm 0.019$	-0.020 ± 0.023	$+0.037 \pm 0.062$	-0.011 ± 0.023	$+0.000 \pm 0.006$	-0.001 ± 0.004
`fgs_attack` (attack-calibrated)	$+0.001 \pm 0.020$	-0.022 ± 0.023	$+0.008 \pm 0.070$	-0.017 ± 0.023	-0.017 ± 0.013	-0.012 ± 0.009
`fgs_hybrid` (hybrid)	—		+0.027 (attack only)	*see caption*		—
`fgs_noproj` (no-proj.)	-0.014 ± 0.026	-0.048 ± 0.019	-0.011 ± 0.072	-0.053 ± 0.019	-0.010 ± 0.031	-0.014 ± 0.025

over-forgets severely (attack gap -0.085, $p < 0.001$, $d_z = -14.2$), pushing forget-edge scores well below the retrain reference. Unlinking alone (UtU) is not enough: attack gaps of +0.058 on Cora and +0.105 on CiteSeer (both $p < 0.001$, $d_z = 2.72/1.94$) confirm the blast-radius hypothesis—the parameters' residual memory of the deleted edges survives structural deletion. (On PubMed the unlink-only gap is just +0.009, so structural deletion does most of the work there.) Paired head-to-head (same seeds, FGS−GNNDelete attack-gap difference), FGS is closer to retrain on Cora (-0.014, $p = 0.034$, $d_z = -0.79$, 95% CI $[-0.027, -0.001]$) and CiteSeer (-0.015, $p = 0.044$, $d_z = -0.74$, 95% CI $[-0.030, -0.001]$), while on PubMed the sign reverses because GNNDelete over-forgets ($+0.087$, $p < 0.001$, $d_z = +10.9$, 95% CI $[+0.081, +0.093]$). The Cora/CiteSeer differences are marginal ($p > 0.01$, uncorrected, CIs abutting zero) and would not survive multiple-comparison correction over the ∼15 paired tests reported here; we state them as directional, not decisive, and the λ sweep below shows they do not hold at tuned λ.

GNNDelete λ sensitivity.

A λ sensitivity sweep ($\lambda \in \{0.25, 0.5, 1, 2, 4\}$, ten seeds, identical eval sets; script archived with the reproduction package) confirms the deletion-vs-neighborhood tradeoff was not tuned at the fixed $\lambda = 1.0$ of Table 4: the best-validation point (smallest $|\Delta_{\text{atk}}|$ with no material utility loss) is $\lambda = 0.5$ on Cora (attack gap -0.005 ± 0.024, utility -0.019 ± 0.022) and CiteSeer ($+0.011 \pm 0.055$, utility -0.005 ± 0.021), and $\lambda = 4.0$ on PubMed (-0.013 ± 0.004, utility $+0.002 \pm 0.002$)—shrinking the reported PubMed over-forget from -0.085 to -0.013 (Table 5). This selection is post-hoc (minimum over five λ values on the same ten seeds), so the "$-0.085 \rightarrow -0.013$" improvement is a selected best, not a prespecified operating point; Table 4 keeps the fixed-λ port for comparability with the other untuned baselines; we flag the port-fidelity limitation in Section 8.

Table 5 GNNDelete neighborhood-influence weight λ sensitivity, 5% deletion, ten seeds: attack gap to retrain (mean ± SD). Larger λ strengthens the neighborhood-influence pull toward the base model's scores, reducing forgetting. Bold: best-validation point per dataset. The $\lambda = 1.0$ column is an independent re-run of the Table 4 GNNDelete setting and reproduces that value within 0.001 (re-run noise).

	$\lambda = 0.25$	$\lambda = 0.5$	$\lambda = 1.0$	$\lambda = 2.0$	$\lambda = 4.0$
Cora	-0.041 ± 0.027	$\mathbf{-0.005 \pm 0.024}$	$+0.022 \pm 0.022$	$+0.038 \pm 0.021$	$+0.048 \pm 0.021$
CiteSeer	-0.047 ± 0.059	$\mathbf{+0.011 \pm 0.055}$	$+0.052 \pm 0.054$	$+0.078 \pm 0.054$	$+0.091 \pm 0.054$
PubMed	-0.236 ± 0.013	-0.158 ± 0.009	-0.086 ± 0.005	-0.037 ± 0.004	$\mathbf{-0.013 \pm 0.004}$

2. The projection causally preserves utility.

The no-projection ablation `fgs_noproj` runs the identical code path with the projection removed ($g = g_f$) and the same attack-calibrated stopping as `fgs_attack`, so the paired `fgs_attack` − `fgs_noproj` comparison isolates the projection: the only difference between the two runs is the orthogonal projection. The projected variant preserves substantially more utility: the paired utility difference is +0.027 on Cora ($p < 0.001$, $d_z = 2.95$) and +0.036 on CiteSeer ($p < 0.001$, $d_z = 4.29$); on PubMed, there is no detectable difference (+0.002, $p = 0.83$), where even the unprojected run preserves utility. Against retrain, the no-projection variant sits −0.048 lower on Cora and −0.053 on CiteSeer (Table 4). Forgetting is comparable with or without the projection (paired attack differences +0.015 to +0.019 on Cora/CiteSeer), but the utility cost of the unprojected run is unacceptable—exactly the tradeoff the method is designed to avoid. One caveat: this ablation uses the attack-calibrated stopping rule, which the Limitations (Section 8) show is auditor-circular, so the projection's causal benefit is demonstrated under that rule; a KS-stopped ablation would be the cleaner causal test.

3. Stopping rules reorder the frontier.

Plain KS (`fgs`) is fastest; the margin guard (`fgs_margin`) shows no detectable difference from plain KS at $n = 10$ (paired gaps differ by < 0.001 on all datasets; vs. retrain, $p = 0.23$, $d_z = 0.40$ on Cora), i.e., the guard essentially never binds and the "fix" is a null result we report as such. The attack-calibrated rule (`fgs_attack`) substantially closes the CiteSeer attack gap *under the evaluation auditor* (paired `fgs` − `fgs_attack` difference +0.029, $p < 0.001$, $d_z = 1.89$) but costs 0.49–0.74× retraining time (median 0.63×), sacrificing most of the speed advantage, and over-forgets on PubMed (attack gap −0.017, $p = 0.003$, $d_z = -1.27$). The hybrid (`fgs_hybrid`) captures a significant share of the attack-calibration gain at near-KS cost: on CiteSeer its mean attack gap is +0.027 vs. +0.037 for KS and +0.008 for the full attack rule (paired hybrid−KS improvement +0.010, $p < 0.001$), at 1.4 s mean selection cost vs. 15–45 s for `fgs_attack` (0.49–0.74× retraining; Table 6).

6.2 Fraction sensitivity: 1%, 10%, 20%

Figure 3 shows attack and utility gaps as functions of the deletion fraction (1%/10%/20% at $n = 5$ seeds each; 5% at $n = 10$ from Table 4). With $n = 5$ per fraction, these are exploratory: 95% confidence intervals are wide, and apparent elevations should be read as hypotheses, not established failure modes. PubMed is flat: attack

Table 6 CiteSeer, 5% deletion, ten seeds: attack gap to retrain for the three stopping rules. The hybrid applies the attack-calibrated key to the five lowest-KS iterates only. Selection cost is the stopping-rule overhead beyond the shared 40-step trajectory: for `fgs` (KS) it is "—" because the rule selects from the trajectory itself with no separate scoring step; for the hybrid it is the CiteSeer-mean cost of scoring five candidates; for `fgs_attack` it is the total wall-clock of the 80-candidate attack-calibrated run (0.49–0.74× retraining on CiteSeer, median 0.63×).

Seed	`fgs` (KS)	`fgs_hybrid`	`fgs_attack`
7	+0.053	+0.048	+0.031
42	−0.005	−0.020	−0.041
123	+0.063	+0.052	+0.031
999	+0.006	−0.005	−0.036
2024	+0.019	+0.016	+0.016
2025	−0.043	−0.051	−0.077
5555	+0.080	+0.076	+0.070
7777	+0.181	+0.171	+0.162
31337	−0.002	−0.021	−0.039
31338	+0.018	+0.004	−0.035
Mean	+0.037	+0.027	+0.008
Selection cost	—	1.4 s	15–45 s

gaps −0.001 to +0.003 and utility within 0.002 across all tested fractions (the 20% attack gap of +0.003 is nominally $p = 0.034$ (Wilcoxon $p = 0.0625$) but negligible in magnitude). Cora shows small negative attack gaps at 1% (-0.017 ± 0.056, 95% CI $[-0.087, +0.052]$) and 10% (-0.017 ± 0.029) but a real utility collapse at 20%: utility -0.055 ± 0.021 ($p = 0.004$; Wilcoxon $p = 0.0625$); the attack gap there (-0.022 ± 0.030, 95% CI $[-0.059, +0.015]$, $p = 0.17$; Wilcoxon $p = 0.31$) is not distinguishable from zero, so the 20% failure is a utility failure, not established over-forgetting. CiteSeer shows modest elevation at 20% ($+0.033 \pm 0.029$, 95% CI $[-0.002, +0.069]$, $p = 0.059$; Wilcoxon $p = 0.13$) with smaller gaps at 1% (+0.009) and 10% (+0.009); the proxy/optimum mismatch diagnosed in Section 6.4 is most visible at higher deletion fractions, but at $n = 5$ this remains suggestive rather than conclusive.

6.3 Efficiency

Wall-clock times on identical 2-CPU hardware (Table 7), measured on the same ten seeds as the accuracy results in a single machine re-run: FGS with KS stopping runs at a median 0.30–0.35× retraining time (0.35×/0.31×/0.30× on Cora/CiteSeer/PubMed), with the expected seed-to-seed variance from adaptive stopping. The GNNDelete baseline is faster still (median $\approx 0.20\times$), reflecting its small operator-training cost. The attack-calibrated variant costs 15–45 s in total run time on CiteSeer (0.49–0.74× retraining, median 0.63×)—roughly two-thirds of retraining time, sacrificing most of the KS variant's speed advantage—while the hybrid adds only 1.4 s mean selection cost.

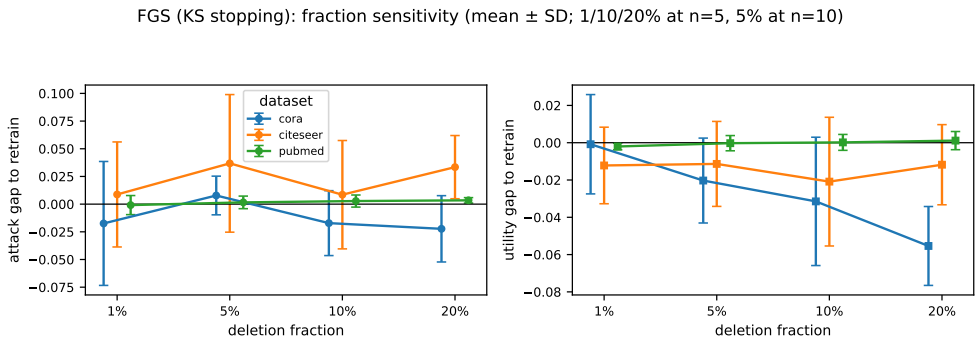


Fig. 3 Attack gap (left) and utility gap (right) to retrain vs. deletion fraction (mean ± SD; 1%/10%/20% at $n = 5$ seeds, 5% at $n = 10$). PubMed is flat; Cora collapses at 20%; CiteSeer is elevated at every fraction. Wilcoxon signed-rank p-values for the $n = 5$ comparisons are reported in the text.

Table 7 Wall-clock time relative to retraining (median over the same 10 seeds as Table 4, single-machine re-run; 2 CPUs). Values < 1 mean faster than retraining. The `fgs_attack` entry reports the median with its full range (seed variance is high for the 80-candidate rule); the other entries are medians. Timing was recorded for the attack-calibrated and hybrid variants on CiteSeer only ("—" elsewhere).

Method	Cora	CiteSeer	PubMed
`fgs` (KS)	0.35×	0.31×	0.30×
GNNDelete	0.20×	0.22×	0.19×
`fgs_attack` (attack-calibrated run)	—	0.63× (range 0.49–0.74×)	—
`fgs_hybrid` (selection only)	—	0.01×	—

The wide range on the attack-calibrated row is seed variance (different seeds settle on different candidates), not uncertainty about the method itself. Timing ratios are hardware-dependent: the ranges above were measured on 2 CPUs with no GPU, and the absolute seconds will shift on different hardware, though the relative ordering of the methods is stable.

6.4 Diagnostics: when the KS proxy diverges

Figure 4 shows the per-iterate KS, attack, and utility trajectories for one configuration per dataset, with the KS-selected and attack-optimal iterates marked. We diagnose the proxy/optimum mismatch systematically, not from a single seed: across all 30 configurations we correlate the KS-minimum iterate with the attack-optimal iterate (minimizing $|\text{attack} - \text{retrain}|$). On CiteSeer the two are uncorrelated (Pearson $r = 0.05$, $p = 0.89$; Spearman $r = -0.00$, $p = 0.99$; mean absolute step difference 8.8)—the KS proxy carries essentially no information about where the attack optimum lies, on any seed. On Cora, they track each other (Pearson $r = 0.62$, $p = 0.058$; Spearman $r = 0.59$, $p = 0.070$; mean $|\Delta| = 4.7$ steps), and on PubMed the relationship is weak (Pearson $r = 0.44$, $p = 0.20$; mean $|\Delta| = 9.4$ steps, inflated by seeds where the KS minimum sits

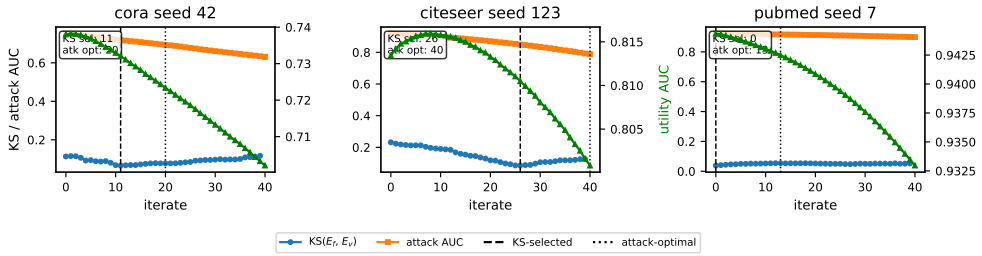


Fig. 4 Per-iterate KS statistic, attack AUC, and utility AUC for one configuration per dataset (Cora seed 42, CiteSeer seed 123, PubMed seed 7). Dashed: KS-selected iterate; dotted: attack-optimal iterate (minimizing |attack − retrain|). On CiteSeer seed 123, the two diverge (26 vs 40), diagnosing the proxy/optimum mismatch; on PubMed seed 7 the KS minimum is at step 0.

at step 0). Pooling across datasets would mix these heterogeneous regimes (CiteSeer $r = 0.05$, Cora $r = 0.62$, PubMed $r = 0.44$), so we report per-dataset values only. The CiteSeer seed-123 trajectories in Figure 4 (KS minimum at step 26, attack optimum at step 40, non-monotone KS) are thus one instance of a dataset-wide pattern, not an anecdote: on CiteSeer the stopping rule optimizes an objective uncorrelated with the attack metric. This proxy/optimum mismatch is our leading explanation for the CiteSeer weakness. The attack-calibrated rule exists precisely for this case, and the hybrid recovers about a third of its benefit at low cost.

6.5 GraphSAGE transfer: a smoke test, not evidence

We ran FGS (KS stopping) on a GraphSAGE backbone for Cora seed 42: attack gap +0.046 and utility gap +0.037 relative to the GraphSAGE retrain, at 0.48× retraining time—qualitatively similar to the GCN behavior (small positive attack gap, preserved utility, 2× speedup). A second configuration timed out on our 2-CPU hardware. With $n = 1$, this is a smoke test that the formulation is not GCN-specific, not a generalization claim; we state it as such.

6.6 Fixed-step ablation: does adaptive stopping help?

We compare the KS-selected iterate against fixed stopping schedules (steps 0, 10, 20, 30, 40) on all 30 configurations, using paired differences of the attack gap to retrain (Table 8). KS-adaptive stopping beats fixed short schedules (steps 0 and 10) and fixed long schedules (steps 30 and 40) decisively (all $p < 0.001$, $|d_z| > 1.0$): stopping too early under-forgets, stopping too late over-forgets. However, *fixed step 20 is statistically tied with KS-adaptive stopping* (paired attack-gap difference −0.0001, $p = 0.97$; utility difference +0.0004, $p = 0.67$). This comparison pools all 30 configurations across the three datasets, mixing the heterogeneous per-dataset regimes diagnosed in Section 6.4; the tie should therefore be read as an aggregate statement, not a per-dataset one.

The conclusion the data support is therefore weaker than "adaptive dominates any fixed schedule": KS stopping is competitive with the best fixed schedule *without*

Table 8 KS-adaptive vs. fixed-step stopping, $n = 30$ configs: paired difference (KS-selected gap minus fixed-step gap) with two-sided t-test. Negative attack difference means KS is closer to retrain.

Fixed step	Attack Δ	p	Utility Δ	p
0	-0.042	< 0.001	-0.006	< 0.001
10	-0.024	< 0.001	-0.005	< 0.001
20	-0.0001	0.97	$+0.0004$	0.67
30	$+0.027$	< 0.001	$+0.007$	< 0.001
40	$+0.056$	< 0.001	$+0.015$	< 0.001

requiring tuning, and it strictly dominates the fixed schedules one would naively choose (too few or too many steps). The obvious objection ("if step 20 ties, why the machinery?") has a direct answer: step 20 was found by looking at the answer. It is close to the median of the KS-selected steps *on these three graphs*, but the per-seed KS selections span 9–29 steps on Cora, 13–28 on CiteSeer, and 0–39 on PubMed—on PubMed seed 7 the right answer is literally zero steps, on others nearly forty. A fixed step chosen without this knowledge risks exactly the failure modes the table shows: stop too early and you under-forget, too late and you over-forget, both decisively worse than the adaptive rule (all $p < 0.001$). The value of the adaptive rule is therefore robustness, not superiority over an oracle fixed schedule: it lands near the per-configuration optimum without a labeled sweep to find it, and it adapts where a fixed schedule cannot (e.g., the PubMed step-0 cases). That is the whole point of the machinery.

7 Discussion: a Pareto frontier, not a winner

Across the sweep, the evidence is a three-way tradeoff among forgetting, utility, and efficiency, with the ranking depending on the dataset, the deletion fraction, and the stopping rule. Every reading in this section is conditional on the tested regime—homophilic citation graphs, a two-layer GCN, a fixed link-inference auditor—and does not extend beyond it. Plain KS stopping is the fast option: near-retrain on PubMed at all tested fractions and on Cora at small fractions, at 0.30–0.35× retraining time (median), but it degrades at 20% on Cora and shows the largest attack gaps on CiteSeer. No single method dominates all three axes. GNNDelete (our unvalidated NumPy port) anchors the speed end of the frontier—the Pareto-optimal speed/utility choice at median 0.19–0.22× vs. 0.30–0.35× retraining time for KS-stopped FGS, and at its post-hoc tuned $\lambda = 0.5$ (selected on these same seeds—an oracle operating point, not a pre-specified one) also the best forgetting choice on Cora and CiteSeer (Section 6.1)—but at the untuned port setting it forgets less well on Cora/CiteSeer and over-forgets on PubMed; fine-tuning does not forget, NegGrad destroys utility, and unlinking alone (UtU) does not reach retrain indistinguishability. Attack-calibrated stopping is the thorough option: it closes the CiteSeer gap under the evaluation auditor (but only under that auditor: a shadow-retrain auditor reopens it, +0.060 vs. +0.008, so we read this family as an auditor-overfitting diagnostic rather than auditor-independent forgetting;

see Limitations) but costs 0.49–0.74× retraining (median 0.63×), sacrificing most of the speed advantage, and over-forgets on PubMed. The hybrid is the compromise: a significant but partial share of the calibration gain (about a third of the KS-to-attack mean improvement) at near-KS cost. No single configuration dominates on all three axes, and we do not claim one does.

Why CiteSeer is hard.

The diagnostics point to a proxy/optimum mismatch: across all ten CiteSeer seeds, the KS-minimum iterate is uncorrelated with the attack-optimal iterate ($r = 0.05$), while the two track each other on Cora ($r = 0.62$) and relate only weakly on PubMed ($r = 0.44$). The CiteSeer attack-gap mean ($+0.037 \pm 0.062$) also deserves a robustness caveat: the median is +0.018, and a single seed (7777, gap +0.181—roughly 3× the next-largest value) drives the mean and most of the variance. Leave-one-out, dropping seed 7777 moves the mean to +0.021 (SD 0.038); dropping any other seed leaves the mean between +0.032 and +0.046. Notably, seed 7777 is also the worst seed for GNNDelete (gap +0.173), so it is a hard data draw, not an FGS-specific artifact. The "largest and most variable" characterization of CiteSeer should therefore be read with this skew in mind: typical seeds sit near +0.02, with one anomalous draw pulling the mean up. A plausible mechanism for the underlying mismatch is CiteSeer's sparser, noisier graph structure, where the validation-edge score distribution is itself a poor stand-in for the retrained forget-edge distribution—but we have not isolated the cause, and we present the mismatch as a diagnosis, not a theorem.

Why Cora collapses at 20%.

At large deletion fractions, the forget set is a substantial part of the training graph; the projected trajectory must travel far, and the first-order protection of Proposition 1 degrades over many steps as the anchor g_r itself moves. The observed utility drop of 0.05–0.06 ($p = 0.004$ at $n = 5$; Wilcoxon $p = 0.0625$) is *suggestive of* the expected failure mode of a local method pushed outside its local regime, but at $n = 5$ this is a hypothesis, not an established failure mode (Section 6.2); the accompanying attack gap (-0.022) is not distinguishable from zero at $n = 5$, so we claim a utility collapse, not over-forgetting. This is a limitation of the method, not of the evaluation.

8 Limitations

1. **No certificate.** FGS is approximate and empirical; we offer no certified-unlearning guarantee, and the fixed non-adaptive auditor may miss residual signal.
2. **Scope.** Edge deletion only, on homophilic citation graphs, with GCN (plus one GraphSAGE smoke test). Node and feature deletion, heterophilic graphs, and larger backbones are untested.
3. **GNNDelete port fidelity.** The main comparison uses a fixed neighborhood-influence weight $\lambda = 1.0$; the deletion-vs-neighborhood tradeoff was not tuned there, and the port was not validated against published GNNDelete edge-deletion numbers. A λ sweep (Section 6.1) shows the tradeoff is real and monotone: the best-validation point is $\lambda = 0.5$ on Cora/CiteSeer and $\lambda = 4.0$ on PubMed, where it shrinks the

reported over-forget from -0.085 to -0.013. The severe PubMed over-forget in Table 4 may therefore reflect the port setting rather than the method.

4. **Stopping is the weak link.** The KS proxy diverges from the attack optimum on CiteSeer; the attack-calibrated rule fixes this only by sacrificing the speed advantage and over-forgetting on PubMed.
5. **Attack calibration is auditor-circular.** The `fgs_attack` rule selects the iterate with the same link-inference auditor family used for evaluation (disjoint negatives, but the same scorer and attack). We tested a mild adaptive auditor on the *same* 80-candidate trajectory that produced Table 4's `fgs_attack` column: a per-seed shadow retrain, stopping at the iterate whose forget-edge scores best match the shadow retrain's (KS). The attack-calibrated choice on this trajectory reproduces the Table 4 value exactly (CiteSeer gap $+0.008 \pm 0.070$), so the comparison is on identical configurations. The shadow-calibrated stop (median step 17) does *not* reproduce the attack-calibrated gap closure (median step 30): its CiteSeer attack gaps are $+0.060 \pm 0.050$ (paired difference $+0.052$, $p = 0.003$, $d_z = 1.27$, $n = 10$)—essentially the plain-KS level. The CiteSeer gap closure should therefore be read as "indistinguishable under this auditor," not as auditor-independent forgetting.
6. **Large fractions.** Utility appears to degrade at 20% deletion on Cora ($n = 5$, exploratory—see Section 6.2); treat FGS as a small-deletion tool pending confirmation at larger n.
7. **Absolute attack AUC stays high.** "Forgotten" means indistinguishable from retraining, not unpredictable—the graph itself leaks link information.
8. **Compute.** All experiments ran on 2 CPUs without a GPU; timings are hardware-specific, though relative comparisons hold.

9 Future Work

The primary open problem is a stopping rule that inherits the KS rule's cost and the attack rule's calibration—the hybrid is a first step, but a principled proxy (e.g., a distributional distance weighted by the attack's ranking geometry) could close the gap. Second, adaptive attackers: evaluating FGS against shadow-model auditors would test whether the retrain gap survives a stronger adversary. Third, node and feature deletion, where the blast-radius analysis suggests the surgery should apply, but the forget gradient needs re-derivation. Finally, the CiteSeer proxy/optimum mismatch deserves a mechanistic explanation.

10 Reproducibility Statement

All experiments use public datasets (Cora, CiteSeer, PubMed) with the splits and seeds documented in Section 5. The experiment code, raw per-seed JSON outputs, and analysis scripts are archived with the submission; every aggregate in this paper is recomputed from the raw JSON by the analysis script, and the audit trail (claim-by-claim recomputation) is included as supplementary material. The archive also includes the verification scripts used in this revision: `scripts/full_rerun.py` (frozen-pipeline re-run establishing seed-pooling exchangeability and same-machine timings),

`scripts/compute_utu.py` (unlink-only baseline), and `scripts/shadow_auditor.py` (shadow-retrain auditor test).

AI Use Disclosure

The author used the AI tool Muse (Muse Spark 1.3, Meta) for drafting and grammatical editing. The contents are reviewed, edited, and attested for accuracy by the author.

A Per-seed tables

Table 9 Per-seed FGS (KS) gaps to retrain, 5% deletion.

Dataset	Seed	Δ_{atk}	Δ_{util}	FGS time (s)	Retrain time (s)
citeseer	7	+0.0526	-0.0340	45.3	117.9
citeseer	42	-0.0053	+0.0170	44.7	116.2
citeseer	123	+0.0629	+0.0046	45.3	115.0
citeseer	999	+0.0061	-0.0330	118.6	230.3
citeseer	2024	+0.0188	-0.0010	84.6	190.0
citeseer	2025	-0.0431	-0.0522	73.8	233.0
citeseer	5555	+0.0795	-0.0218	97.6	329.2
citeseer	7777	+0.1807	+0.0135	110.5	242.0
citeseer	31337	-0.0019	-0.0028	54.2	136.3
citeseer	31338	+0.0178	-0.0039	86.9	212.2
cora	7	+0.0035	-0.0273	14.1	36.0
cora	42	+0.0229	-0.0320	14.7	37.0
cora	123	+0.0220	+0.0274	13.9	37.2
cora	999	-0.0045	-0.0078	33.4	76.4
cora	2024	+0.0043	-0.0001	15.3	41.5
cora	2025	-0.0255	-0.0329	26.9	62.4
cora	5555	+0.0267	-0.0370	28.2	63.6
cora	7777	-0.0037	-0.0305	49.7	104.2
cora	31337	+0.0302	-0.0520	15.8	44.7
cora	31338	+0.0021	-0.0108	27.1	69.9
pubmed	7	+0.0058	+0.0018	34.3	90.9
pubmed	42	-0.0008	-0.0021	46.1	127.9
pubmed	123	-0.0028	-0.0077	33.7	89.5
pubmed	999	+0.0033	-0.0010	117.2	316.6
pubmed	2024	+0.0049	+0.0035	54.9	142.6
pubmed	2025	-0.0027	-0.0038	85.8	226.5
pubmed	5555	-0.0090	-0.0012	66.6	190.8
pubmed	7777	+0.0116	+0.0069	116.6	323.6
pubmed	31337	+0.0018	-0.0011	54.4	146.9
pubmed	31338	+0.0037	+0.0023	77.5	225.7

B Stopping-rule trajectories

Per-iterate KS, attack, and utility trajectories for the three diagnostic configurations are plotted in Figure 4; the underlying arrays (41 iterates each) are archived in `results/fixedstep.json` under `configs` (keys `ks`, `attack`, `util`) with per-config provenance (cache file, JSON file, and RNG streams).

C Claim-by-claim audit trail

The headline quantitative claims in Sections 6.1–6.6 are checked against the raw per-seed JSONs (`reproduction-package/results/per_seed/*.json`) by the verifier `reproduction-package/scripts/verify_reproduction.py`, which recomputes every headline number from the raw outputs and exits nonzero on any mismatch; expected values are pinned in `reproduction-package/expected/headline_numbers.json`. The fixed-step validation (maximum attack deviation 0.0059 vs. tolerance 0.01) is recorded in `reproduction-package/results/fixedstep.json` under `validation`.

Data and Code Availability

All experiment code, configurations, raw per-seed outputs, and the one-command reproduction package (`bash reproduce.sh`, with the verifier script that re-checks every headline number) are included as supplementary material. We will release a public code repository upon acceptance.

Statements and Declarations

Competing Interests

The author declares no competing interests. No funding supported this work.

References

[1] Bourtoule, L., Chandrasekaran, V., Choquette-Choo, C.A., Jia, H., Travers, A., Zhang, B., Lie, D., Papernot, N.: Machine unlearning. In: Proc. IEEE Symp. Security and Privacy (SP), pp. 141–159 (2021)

[2] Ginart, A., Guan, M., Valiant, G., Zou, J.Y.: Making AI forget you: Data deletion in machine learning. In: Adv. Neural Inf. Process. Syst. 32 (2019)

[3] Chen, M., Zhang, Z., Wang, T., Backes, M., Humbert, M., Zhang, Y.: Graph unlearning. In: Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS), pp. 499–513 (2022)

[4] Said, A., Zhao, Y., Derr, T., Shabbir, M., Abbas, W., Koutsoukos, X.: A survey of graph unlearning. arXiv:2310.02164 (2023)

[5] Jain, S., Kalvani, R., Galhotra, S., Ranu, S.: Is graph unlearning ready for practice? A benchmark on efficiency, utility, and forgetting. In: Proc. Int. Conf. Learning Representations (ICLR) (2026)

[6] Tan, J., Sun, F., Qiu, R., Su, D., Shen, H.: Unlink to unlearn: Simplifying edge unlearning in GNNs. In: Companion Proc. ACM Web Conf. (WWW) (2024)

[7] Yu, T., Kumar, S., Gupta, A., Levine, S., Hausman, K., Finn, C.: Gradient surgery for multi-task learning. In: Adv. Neural Inf. Process. Syst. 33 (2020)

[8] Zhu, S., Zhao, H., Wu, T., Wang, P., Deng, H., Xu, J., Zheng, B.: Gradient deconfliction via orthogonal projections onto subspaces for multi-task learning. In: Proc. ACM Int. Conf. Web Search and Data Mining (WSDM) (2025)

[9] Hoang, T., Rana, S., Gupta, S., Venkatesh, S.: Learn to unlearn for deep neural networks: Minimizing unlearning interference with gradient projection. In: Proc. IEEE/CVF Winter Conf. Applications of Computer Vision (WACV), pp. 4819–4828 (2024)

[10] Zhou, D., Zhang, Y., Wei, T., Qiu, R., Yang, K., Lin, X., Qian, C., He, J., Tong, H., Zhai, C., Ji, H., Zhang, H.: Geometric-disentangelment Unlearning. arXiv:2511.17100. v1 Nov 2025; v4 Feb 2026. Title spelling as on arXiv. (2025)

[11] Feng, Q., Tu, J., Kang, M., Zhao, H., Zhang, C., Qian, H.: FG-OrIU: Towards Better Forgetting via Feature-Gradient Orthogonality for Incremental Unlearning. arXiv:2601.13578. v1 Jan 2026. Vision pre-trained models, incremental class unlearning; gradient orthogonal projection. (2026)

[12] Gennaro, F.D., Shevchenko, A., Yang, F.: Fast Unlearning at Scale via Margin Self-Correction. arXiv:2606.02920. LLM unlearning with an online stopping rule. (2026)

[13] Fan, B., Ai, Y., Li, X., Guo, Z., Li, R.-H., Wang, G.: OpenGU: A comprehensive benchmark for graph unlearning. arXiv:2501.02728 (2025)

[14] Wang, C.-L., Huai, M., Wang, D.: Inductive graph unlearning. In: Proc. 32nd USENIX Security Symp., pp. 3205–3222 (2023)

[15] Wu, J., Yang, Y., Qian, Y., Sui, Y., Wang, X., He, X.: GIF: A general graph unlearning strategy via influence function. In: Proc. ACM Web Conf. (WWW), pp. 651–661 (2023)

[16] Cheng, J., Dasoulas, G., He, H., Agarwal, C., Zitnik, M.: GNNDelete: A general strategy for unlearning in graph neural networks. In: Proc. Int. Conf. Learning Representations (ICLR) (2023). arXiv:2302.13406

[17] Li, X., Zhao, Y., Wu, Z., Zhang, W., Li, R.-H., Wang, G.: Towards effective and

general graph unlearning via mutual evolution. In: Proc. AAAI Conf. Artificial Intelligence, vol. 38, pp. 13682–13690 (2024)

[18] Shen, C., Wang, M., Zhang, J.: SURGE: Structural perturbation response field guided residual distillation for graph unlearning. Machine Learning **115**(10) (2026) https://doi.org/10.1007/s10994-026-07176-x

[19] Cong, W., Mahdavi, M.: Efficiently forgetting what you have learned in graph representation learning via projection. In: Proc. Int. Conf. Artificial Intelligence and Statistics (AISTATS). PMLR, vol. 206, pp. 6674–6703 (2023)

[20] Yang, Z.-R., Han, J., Wang, C.-D., Liu, H.: Erase then rectify: A training-free parameter editing approach for cost-effective graph unlearning. arXiv:2409.16684 (2024)

[21] Kolipaka, V., Sinha, A., Mishra, D., Kumar, S., Arun, A., Goel, S., Kumaraguru, P.: A cognac shot to forget bad memories: Corrective unlearning for graph neural networks. In: Proc. Int. Conf. Machine Learning (ICML) (2025)

[22] Levi, A., Kontorovich, G., Shimkin, N.: Go beyond your means: Unlearning with per-sample gradient orthogonalization. arXiv:2503.02312 (2025)

[23] He, X., Jia, J., Backes, M., Gong, N.Z., Zhang, Y.: Stealing links from graph neural networks. In: Proc. 30th USENIX Security Symp., pp. 2669–2686 (2021)

[24] Dong, Y., Zhang, B., Lei, Z., Zou, N., Li, J.: IDEA: A flexible framework of certified unlearning for graph neural networks. In: Proc. 30th ACM SIGKDD Conf. Knowledge Discovery and Data Mining, pp. 621–630 (2024)

[25] D'Angelo, A., Gullo, F., Mottin, D., Ranu, S., Stilo, G.: Lethe: Link inference attacks for evaluation of edge unlearning methods. In: Proc. NeurIPS 2026, Experiments & Benchmarks Track (2026). To appear; arXiv:2610.21869

[26] Patel, G., Qiu, Q.: Learning to unlearn while retaining: Combating gradient conflicts in machine unlearning. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 4211–4221 (2025). Also arXiv:2503.06339. Implicit gradient regularization to avoid forget/retain gradient conflicts.

[27] Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks. In: Proc. Int. Conf. Learning Representations (ICLR) (2017)

[28] Massey, F.J.: The Kolmogorov–Smirnov test for goodness of fit. J. Amer. Stat. Assoc. **46**(253), 68–78 (1951)

[29] Shokri, R., Stronati, M., Song, C., Shmatikov, V.: Membership inference attacks against machine learning models. In: Proc. IEEE Symp. Security and Privacy

(SP), pp. 3–18 (2017)

[30] Yang, Z., Cohen, W.W., Salakhutdinov, R.: Revisiting semi-supervised learning with graph embeddings. In: Proc. Int. Conf. Machine Learning (ICML), pp. 40–48 (2016)